

TRANSLATED BLOG / 译文

构建高效的 agent

来自 Anthropic 工程团队的实践经验：从增强型 LLM、workflow，到自主 agent

作者：Erik S.、Barry Zhang

发布日期：2024-12-19

原文：<https://www.anthropic.com/engineering/building-effective-agents>

中文翻译 · DARK READING EDITION

<https://www.anthropic.com/engineering/building-effective-agents>

过去一年，我们与数十支跨行业团队一起构建大语言模型agent。我们反复观察到：最成功的实现通常并没有采用复杂框架或专用库，而是使用简单、可组合的模式。本文分享我们从客户项目和自身实践中总结出的经验，并为开发者提供构建高效agent的实用建议。

什么是agent？

“agent”可以有多种定义。有些客户把它理解为能在较长时间里独立运行、调用多种工具完成复杂任务的全自主系统；另一些人则用这个词指代沿预先定义workflow执行的、更具约束性的实现。Anthropic 将这些都归入“agent系统”，但会在架构上明确区分workflow与agent：

- workflow: LLM 和工具按照预先写好的代码路径被编排。
- agent: LLM 动态决定自己的执行过程与工具使用方式，并掌握如何完成任务。

下文会详细讨论这两类系统。附录一还会介绍两种已经体现出突出价值的实际应用领域。

什么时候该用agent，什么时候不该用？

构建 LLM 应用时，我们建议先寻找最简单的可行方案，只在确有必要时增加复杂度。这甚至可能意味着完全不构建agent系统。agent系统常常以更高的延迟和成本换取更好的任务表现，因此应先判断这种交换是否值得。

如果复杂度确有必要：对于定义明确的任务，workflow更可预测、更一致；当系统需要灵活性，并要让模型大规模地自主决策时，agent更合适。不过，对许多应用来说，优化一次 LLM 调用，再配合检索和上下文示例，通常已经足够。

什么时候以及如何使用框架

目前有很多能简化agent系统开发的框架，例如 Claude Agent SDK、AWS 的 Strands Agents SDK，以及 Rivet、Vellum 这类可视化workflow工具。

框架会替你处理调用模型、定义与解析工具、串联多次调用等底层工作，因此很容易上手。但它们也会增加抽象层，遮蔽真正的提示词和响应，使调试变难；同时还容易诱导开发者加入原本并不需要的复杂度。

我们的建议是：一开始直接使用 LLM API。许多模式用几行代码就能实现。如果确实使用框架，也要看懂它背后的代码。对底层行为作出错误假设，是客户项目中很常见的问题来源。

构件、workflow与agent

下面从agent系统最基础的构件开始，逐步增加复杂度：先是简单的组合式workflow，最后才是自主agent。

基础构件：增强型 LLM

agent系统最基础的构件，是通过检索、工具和记忆等能力增强的 LLM。当前模型已经可以主动使用这些能力：生成自己的检索查询、选择合适的工具，并判断哪些信息值得保留。



实现时应重点关注两点：一是根据具体用例定制这些能力；二是为 LLM 提供简单、清晰、有完整文档的接口。实现增强能力的方式很多，其中一种是 Model Context Protocol (MCP)：开发者通过一个简单客户端，就能接入不断扩大的第三方工具生态。下文默认每次 LLM 调用都能够使用这些增强能力。

workflow：提示链

提示链把任务拆成一系列步骤，每次 LLM 调用处理上一步的输出。你还可以在任何中间步骤加入程序检查（即“闸门”），确认执行仍在正确轨道上。



适用场景：任务能够轻松、清楚地分解为固定子任务。它的核心取舍是牺牲延迟来提高准确率，因为每次调用面对的任务都更简单。

- 先生成营销文案，再把它翻译成另一种语言。
- 先写文档提纲，检查提纲是否符合标准，再据此撰写正文。

workflow: 路由

路由先对输入分类，再把它交给专门的后续任务。这样可以分离不同职责，并为每类输入编写更有针对性的提示词。否则，针对某一类输入的优化可能会损害其他输入的表现。



适用场景：复杂任务包含若干明显不同的类别，而且 LLM 或传统分类模型/算法能够准确完成分类。

- 把不同客服问题（一般咨询、退款、技术支持）分别交给不同流程、提示词和工具。
- 把简单常见的问题交给成本较低的小模型，把困难或罕见的问题交给能力更强的模型，以优化整体效果和成本。

workflow: 并行化

LLM 有时可以同时处理一项任务的不同部分，再由程序汇总结果。并行化主要有两种形式：

- 分区：把任务拆成彼此独立、可并行执行的子任务。
- 投票：多次执行同一任务，以获得多样化结果。

适用场景：子任务可并行以提升速度，或需要多个视角、多个尝试来提高可信度。面对包含多项考量的复杂任务，让每次 LLM 调用只专注一个方面，通常比让一次调用同时处理所有方面效果更好。

分区示例：一个模型处理用户问题，另一个模型专门检查是否包含不当内容；或在自动评测中，让不同调用分别评价模型表现的不同方面。

投票示例：使用多个不同提示词审查同一段代码的漏洞；或让多个调用从不同维度判断内容是否不当，并设置不同票数阈值，以平衡误报和漏报。

workflow: 编排者 - 工作节点

在“编排者 - 工作节点”workflow中，一个中心 LLM 动态拆解任务，把子任务分派给多个工作 LLM，最后综合它们的结果。



适用场景：无法预先判断需要哪些子任务的复杂工作。例如代码任务要改多少文件、每个文件怎样改，都取决于具体请求。它与并行化在形态上相似，但关键差异是灵活性：子任务不是预先定义的，而是由编排者根据输入动态决定。

- 每次都可能需要跨多个文件做复杂修改的编程产品。
- 需要从多个来源收集、筛选并分析潜在相关信息的搜索任务。

workflow: 评估者 - 优化者

在“评估者 - 优化者”workflow中，一次 LLM 调用生成答案，另一次调用负责评价和反馈，二者循环迭代。



适用场景：存在清楚的评价标准，而且迭代修改能带来可衡量的价值。两个重要信号是：第一，人类给出反馈后，LLM 的回答确实能明显改善；第二，LLM 自己也能给出这种有效反馈。这类似人类作者为写出成熟作品而反复修改的过程。

- 文学翻译：初次译文可能遗漏细微含义，而评估模型能提出有效批评。
- 复杂搜索：需要多轮搜索与分析，评估模型决定是否还应继续检索。

agent

随着 LLM 在理解复杂输入、推理与规划、可靠使用工具、从错误中恢复等关键能力上逐渐成熟，agent开始进入生产环境。agent通常从用户的一条命令或一段互动讨论开始。任务明确后，它会自行规划和执行，必要时再回来请求用户补充信息或作出判断。

执行期间，agent必须在每一步从环境中获得“事实依据”，例如工具调用结果或代码执行结果，以判断自己的进度。它可以在检查点或遇到阻碍时暂停，请人类提供反馈。任务通常在完成时终止，也常设置最大迭代次数等停止条件，以维持控制。

agent虽然能处理复杂任务，实现方式却往往很直接：本质上就是一个 LLM 根据环境反馈，在循环中决定并使用工具。因此，工具集及其文档必须经过清晰、审慎的设计。



适用场景：开放式问题的步骤数难以或不可能预先判断，也无法硬编码固定路径。LLM 可能执行很多轮，因此你必须对它的决策能力有一定信任。**agent**的自主性很适合在可信环境中扩展任务规模，但也意味着更高成本和错误累积的风险。应在沙盒环境中进行充分测试，并设置适当的护栏。

- 解决 SWE-bench 任务的编程**agent**：根据任务描述修改多个文件。
- “电脑使用”参考实现：Claude 操作计算机以完成任务。

组合与定制这些模式

这些构件不是强制规范，而是常见模式。开发者应根据用例调整 and 组合它们。成功的关键与其他 LLM 功能一样：衡量表现，并持续迭代实现。再次强调，只有在复杂度能够被证明会改善结果时，才应加入它。

总结

在 LLM 领域，成功并不意味着构建最复杂的系统，而是构建最适合自己需求的系统。先从简单提示词开始，通过全面评测优化它；只有当简单方案不够用时，再加入多步骤 **agent** 系统。

实现**agent**时，我们遵循三项核心原则：

- 保持**agent**设计简单。
- 优先保证透明度，明确展示**agent**的规划步骤。
- 通过完整的工具文档和测试，精心设计**agent** - 计算机接口（ACI）。

框架能帮助你快速开始，但进入生产阶段时，不必害怕减少抽象层、回到基础组件。遵循上述原则，才能构建既强大，又可靠、可维护、值得用户信任的**agent**。

致谢

本文由 Erik S. 与 Barry Zhang 撰写，内容来自 Anthropic 构建**agent**的经验，以及客户分享的宝贵实践。

附录一：agent的实际应用

客户实践显示，AI agent在两个领域尤其有前景。两者共同说明：当任务同时需要“对话”和“行动”，有清楚的成功标准，允许反馈循环，并包含有意义的人类监督时，agent最能创造价值。

A. 客户支持

客户支持把熟悉的聊天界面与工具集成带来的增强能力结合起来，非常适合更开放的agent：

- 支持过程天然以对话展开，同时需要访问外部信息并执行操作。
- 工具可用于获取客户资料、订单历史和知识库文章。
- 退款、更新工单等动作可以由程序执行。
- 能用用户定义的问题解决结果清楚衡量成功与否。

已有一些公司采用按成功解决次数计费的模式，只有问题真正解决才收费，这也体现了它们对agent效果的信心。

B. 编程agent

软件开发领域已经展现出 LLM 的巨大潜力：能力从代码补全不断演进到自主解决问题。agent尤其有效，因为：

- 代码方案能通过自动化测试验证。
- agent可把测试结果作为反馈，反复迭代方案。
- 问题空间定义明确且结构化。
- 输出质量可以客观衡量。

在 Anthropic 的实现中，agent如今只凭拉取请求描述，就能解决 SWE-bench Verified 基准中的真实 GitHub Issue。尽管自动测试有助于验证功能，人类审查仍不可缺少，因为还要确认方案符合系统的更广泛要求。

附录二：为工具做提示工程

无论构建哪种agent系统，工具都很可能是重要组成部分。工具通过 API 中精确的结构和定义，让 Claude 能与外部服务及 API 交互。当 Claude 决定调用工具时，其 API 响应会包含工具使用区块。工具定义和规格值得投入与总体提示词同等程度的提示工程。

同一个动作往往能用不同方式表达。例如，编辑文件可以写 `diff`，也可以重写整个文件；结构化输出可以把代码放在 `Markdown` 中，也可以放在 `JSON` 中。从软件工程角度看，这些格式差异多半只是表面差异，可以无损转换，但对 LLM 来说，有些格式明显更难生成。

例如，写 `diff` 时必须在新代码尚未全部写出前，就准确知道区块头里会改变多少行；把代码放进 `JSON`，则必须额外转义换行和引号。

选择工具格式时，建议：

- 给模型足够的 `token` 先“思考”，避免它过早把自己逼进死角。
- 让格式尽量接近模型在互联网自然文本中经常见到的形式。
- 避免格式性负担，例如精确统计数千行代码，或为整段代码转义字符串。

一个经验法则是：人们在设计人机界面（HCI）上投入多少精力，就应在设计agent - 计算机接口（ACI）上投入同等精力。

- 站在模型角度思考：仅凭描述和参数，工具用法是否显而易见？好的工具定义通常包含使用示例、边界情况、输入格式要求，以及它与其他工具의 清楚边界。
- 通过调整参数名或描述，让含义更直接。可以把它想成给团队里的初级开发者写一份优秀 `docstring`；当工具很多且相似时，这一点尤其重要。
- 测试模型如何使用工具：用大量示例输入观察它会犯什么错，再持续改进。
- 对工具做“防错设计”：修改参数和接口，让错误更难发生。

在构建 `SWE-bench agent`时，Anthropic 实际上花在优化工具上的时间，比优化总体提示词还多。例如，agent离开仓库根目录后，使用相对路径的工具会出错。团队于是把工具改成始终要求绝对路径，模型随后能够稳定、无误地使用它。

原文链接：<https://www.anthropic.com/engineering/building-effective-agents>